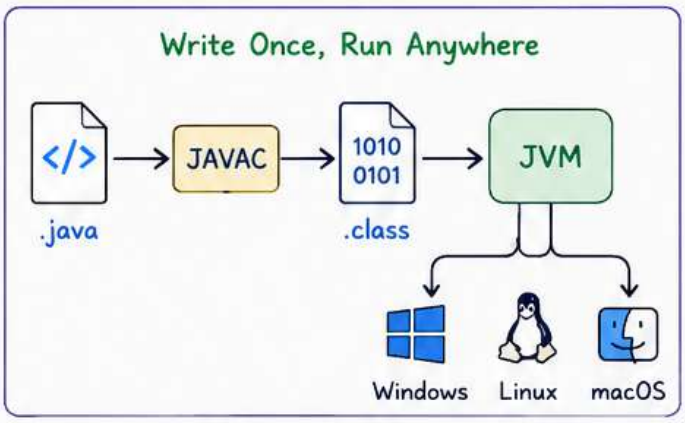
JAVA NOTES

1 INTRODUCTION TO JAVA

- Java is a high-level, class-based, object-oriented programming language.
- Developed by Sun Microsystems (1995).
- Platform Independent – "Write Once, Run Anywhere"
- Secure, Robust, Portable, Multithreaded and Architecture Neutral.
- Java applications are compiled to bytecode (.class) which runs on JVM.



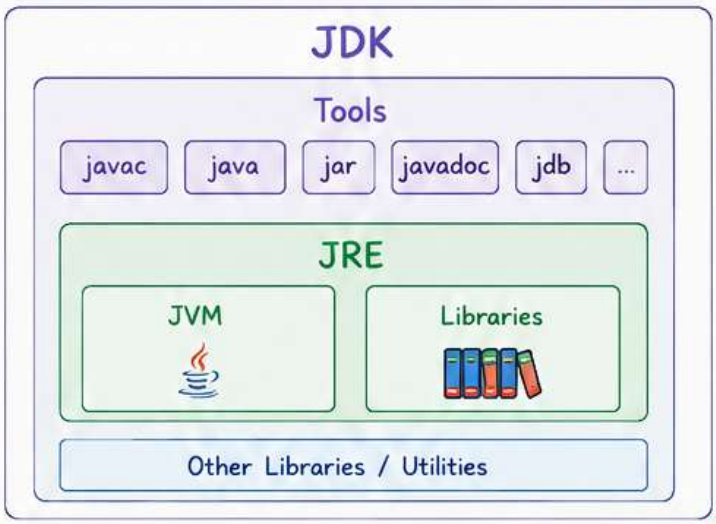
2 JDK (JAVA DEVELOPMENT KIT)

JDK is a software development kit used to develop Java applications.

Includes:

- JRE
- javac (Java Compiler)
- java (Launcher)
- Tools (jar, javadoc, jdb, etc.)
- Libraries and other utilities

Purpose:
Provides everything required to write, compile, debug and run Java programs.



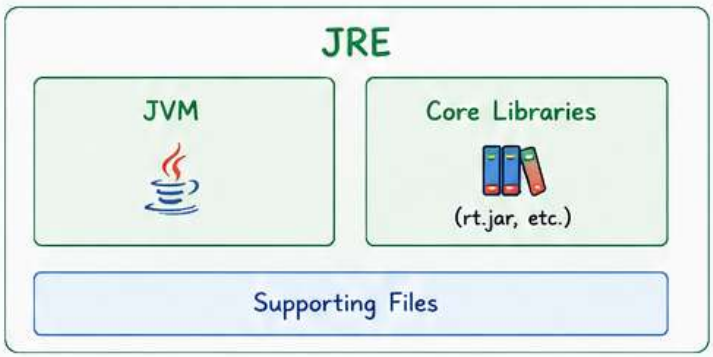
3 JRE (JAVA RUNTIME ENVIRONMENT)

JRE provides the environment to run Java applications.

Includes:

- JVM
- Core Libraries
- Other supporting files

Purpose:
Allows you to run Java programs.
(Does NOT include development tools like javac)



4 JVM (JAVA VIRTUAL MACHINE) – INTERNAL WORKING

JVM is the heart of Java. It executes bytecode and provides runtime environment.

Execution Flow Inside JVM

1. Class Loader Subsystem	2. Bytecode Verifier	3. Execution Engine	4. Runtime Data Areas	5. Garbage Collector
Loads .class files into memory.	Verifies code for safety and security.	Executes bytecode using the Runtime Data Areas and JVM TI.	Stores data required during program execution.	Automatically reclaims memory no longer in use.

Runtime Data Areas

Method Area (Class Information, Static Variables, Constants)	Heap Area (Objects and Arrays)	Java Stacks (One per Thread)	PC Register (Program Counter for each Thread)	Native Method Stack (For Native Method Execution)

Key Points:

✓ JVM is platform specific. ✓ It converts bytecode into machine code. ✓ It enables security, memory management and portability.

CHAPTER 3 — Java Basics & Syntax

1. What is Java?

- Java is a simple, object-oriented, platform-independent, secure and robust programming language.
- Write Once, Run Anywhere (WORA)

2. Structure of a Java Program

```
1  // This is a simple Java program
2  public class HelloWorld {
3      public static void main(String[] args) {
4          // This is a print statement
5          System.out.println("Hello, World!");
6      }
7  }
8
9
```

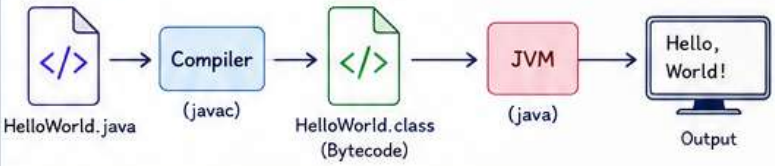
3. Explanation

Line	Description
2	`class` keyword is used to declare a class.
3	`main()` is the entry point of every Java program.
3	`public`: accessible everywhere. `static`: belongs to the class, not an object. `void`: does not return any value. `String[] args`: command line arguments.
5	`System.out.println()` prints output on the screen.

4. Basic Syntax Rules

- Java is case-sensitive (class and Class are different).
- Every statement must end with a semicolon (;).
- Code blocks are written inside { }.
- The file name must be same as the class name and saved with .java extension.
- One public class per file.

5. Compilation & Execution Process



Steps:

1. Write the code in a .java file.
2. Compile it using: `javac HelloWorld.java`
3. This generates bytecode (HelloWorld.class).
4. JVM executes the bytecode using: `java HelloWorld`
5. Output is displayed on the screen.

6. Comments in Java

```
// This is a single line comment
/*
   This is a
    multi-line comment
*/
```

7. Keywords (Examples)

class, public, static, void, int, double, if, else, for, while, return, new, this, extends, implements, try, catch, finally, break, continue

CHAPTER 4 — Variables & Data Types

Tip

Use meaningful variable names like age, totalMarks, isValid, not a, x, y.

1. Variables

- Variables are containers that store data values.
- In Java, every variable must be declared before use.
- Syntax:

```
dataType variableName = value;
```

Example:

```
int age = 20;
double salary = 55000.50;
char grade = 'A';
boolean isJavaFun = true;
```

2. Data Types in Java

Java has two types of data types:

- Primitive Data Types (built-in)
- Non-Primitive / Reference Data Types

3. Primitive Data Types

Type	Size	Range	Default Value	Example
byte	1 byte	-128 to 127	0	`byte b = 10;`
short	2 bytes	-32,768 to 32,767	0	`short s = 1000;`
int	4 bytes	-2^{31} to $2^{31}-1$	0	`int age = 25;`
long	8 bytes	-2^{63} to $2^{63}-1$	0L	`long big = 100000L;`
float	4 bytes	~3.4E-38 to 3.4E+38	0.0f	`float f = 10.5f;`
double	8 bytes	~1.7E-308 to 1.7E+308	0.0d	`double d = 99.99;`
char	2 bytes	\u0000 to \uffff	'\u0000'	`char ch = 'Z';`
boolean	1 bit*	true or false	false	`boolean flag = true;`

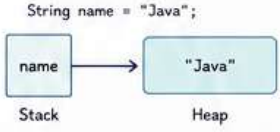
* Size of boolean is JVM dependent.

4. Reference (Non-Primitive) Data Types

These store references (addresses) to objects.

Examples:

- String
- Arrays
- Classes
- Interfaces
- Enums



5. Type Casting

- Automatic (Widening): smaller → larger

  `int → long → float → double`

- Manual (Narrowing): larger → smaller

  Requires explicit casting.

Example:

```
int a = 10;
double d = a;          // automatic
double x = 10.5;
int y = (int)x;        // manual
```

6. Naming Rules for Variables

- ✓ Must start with a letter, _ or $
- ✓ Cannot start with a number
- ✓ Can contain letters, digits, _ or $
- ✓ Case-sensitive (age, Age and AGE are different)
- ✓ Cannot use reserved keywords

7. Constant (final) Variables

- Use final keyword to create constants.
- Value cannot be changed.
- Example:

```
final double PI = 3.14159;
// PI = 3.14;    // Error
```

8. Default Values for Instance Variables

If not initialized, instance variables get default values.

Examples: int → 0, boolean → false, char → '\u0000', double → 0.0, Object → null

Key Takeaway Variables store data.
Data types define the kind of data.
Choose the right type to save memory and avoid errors.

CHAPTER 5 — Operators & Control Statements

1. Operators in Java

A. Arithmetic Operators

Operator	Description	Example (a = 10, b = 3)
+	Addition	a + b = 13
-	Subtraction	a - b = 7
*	Multiplication	a * b = 30
/	Division	a / b = 3
%	Modulus	a % b = 1
++	Increment	a++ = 11
--	Decrement	a-- = 9

B. Relational Operators

Operator	Description	Example (a = 10, b = 20)
==	Equal to	a == b → false
!=	Not equal	a != b → true
>	Greater than	a > b → false
<	Less than	a < b → true
>=	Greater or equal	a >= b → false
<=	Less or equal	a <= b → true

C. Logical Operators

Operator	Description	Example (A=true, B=false)
&&	Logical AND	A && B → false
||	Logical OR	A || B → true
!	Logical NOT	!A → false

D. Assignment Operators

Operator	Description	Example (a = 10)
=	Assign	a = 10
+=	Add & assign	a += 5 → 15
-=	Sub & assign	a -= 5 → 5
*=	Mul & assign	a *= 2 → 20
/=	Div & assign	a /= 2 → 5
%=	Mod & assign	a %= 3 → 1

2. Control Statements

A. if Statement

```
int age = 18;
if (age >= 18) {
    System.out.println("Adult");
}
```

B. if-else Statement

```
int age = 16;
if (age >= 18) {
    System.out.println("Adult");
} else {
    System.out.println("Minor");
}
```

C. if-else if-else Ladder

```
int marks = 75;
if (marks >= 90) {
    System.out.println("A");
} else if (marks >= 75) {
    System.out.println("B");
} else if (marks >= 60) {
    System.out.println("C");
} else {
    System.out.println("D");
}
```

D. switch Statement

```
int day = 2;
swtich (day) {
  case 1: System.out.println("Mon"); break;
  case 2: System.out.println("Tue"); break;
  case 3: System.out.println("Wed"); break;
  default: System.out.println("Invalid");
}
```

E. for Loop

```
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");
}
```

Output: 1 2 3 4 5

F. while Loop

```
int i = 1;
while (i <= 5) {
    System.out.print(i + " ");
    i++;
}
```

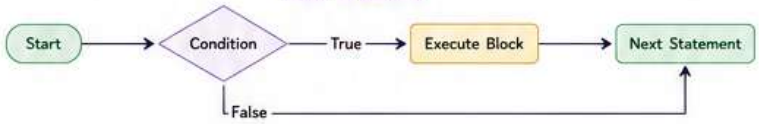
Output: 1 2 3 4 5

G. do-while Loop

```
int i = 1;
do {
    System.out.print(i + " ");
    i++;
} while (i <= 5);
```

Output: 1 2 3 4 5

Flow of Control



Key Takeaways

- Operators perform operations on data and variables.
- Control statements decide the flow of execution based on conditions.
- Loops are used to repeat a block of code.

CHAPTER 6 — Arrays & Strings

1. Arrays

- Array is a collection of similar data elements stored in contiguous memory locations.

2. One-Dimensional Array

- Declaration & Initialization

```
int[] numbers = new int[5];     // declaration
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
numbers[3] = 40;
numbers[4] = 50;
```

Memory Representation (1D Array)

Index:	0	1	2	3	4
numbers:	10	20	30	40	50

Address →

- Accessing Elements

```
System.out.println(numbers[0]);  // 10
System.out.println(numbers[4]);  // 50
```

- Traversal

```
for (int i = 0; i < numbers.length; i++) {
    System.out.print(numbers[i] + " ");
}
```

Output: 10 20 30 40 50

3. Multi-dimensional Array

- Declaration & Initialization

```
int[][] matrix = new int[2][3];
matrix[0][0] = 1;
matrix[0][1] = 2;
matrix[0][2] = 3;
matrix[1][0] = 4;
matrix[1][1] = 5;
matrix[1][2] = 6;
```

- Traversal

```
for (int i = 0; i < 2; i++) {
   for (int j = 0; j < 3; j++) {
      System.out.print(matrix[i][j] + " ");
   }
   System.out.println();
}
```

Output:
1 2 3
4 5 6

Key Takeaways

- Arrays store multiple values of the same type.
- Indexing in arrays starts from 0.
- Use arrays when you have multiple values of the same type.

4. Strings

- String is a sequence of characters.
- In Java, String is an object of String class.
- Strings are immutable (cannot be changed).

5. Declaration

```
String name = "Hello";
String msg = new String("Java");
```

6. Commonly Used Methods

Method	Description	Example
length()	Returns length of string	name.length() → 5
charAt(index)	Returns character at index	name.charAt(1) → 'e'
substring(i, j)	Returns substring [i, j)	name.substring(1, 4) → "ell"
equals(s)	Compares content	name.equals("Hello") → true
equalsIgnoreCase(s)	Compares (ignore case)	name.equalsIgnoreCase("hello")
toUpperCase()	Converts to upper case	name.toUpperCase()
toLowerCase()	Converts to lower case	name.toLowerCase()
trim()	Removes leading/trailing spaces	name.trim()

7. String Concatenation

```
String first = "Hello";
String last = "Javaa^";
String full = first + " " + last;
System.out.println(full);  // Hello Java
```

8. Example

```
String s = "  Java Programming  ";
System.out.println(s.trim());
                        // "Java Programming"
System.out.println(s.toUpperCase()));
                // "  JAVA PROGRAMMING  "
System.out.println(s.substring(2, 6));  // "va P"
```

Key Takeaways

- Strings are immutable; any modification creates a new String object.
- Use String methods to perform common operations easily.
- Arrays and Strings are widely used in real-world applications.

CHAPTER 7

OOP Concepts

1. What is OOP?

- OOP (Object-Oriented Programming) is a programming paradigm based on the concept of "Objects".
- Java is a pure object-oriented language.

Benefits of OOP

- ✓ Modularity
- ✓ Reusability
- ✓ Maintainability
- ✓ Scalability
- ✓ Data Security

2. Class & Object

- Class is a blueprint or template.
- Object is an instance of a class.

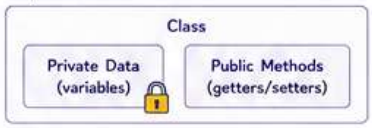
```
class Car {
  String color;
  int speed;
  void display() {
     System.out.println("Color: " + color +
       ", Speed: " + speed);
  }
}
public class Main {
  public static void main(String[] args) {
    Car c1 = new Car();   // object creation
    c1.color = "Red";
    c1.speed = 120;
    c1.display();
  }
}
```

```
Output: Color: Red, Speed: 120
```

3. Four Pillars of OOP

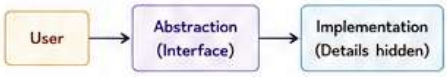
1. Encapsulation (Data Hiding)

- Binding data and methods together in a single unit (class) and restricting direct access to some components.



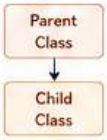
2. Abstraction

- Showing only essential features and hiding the implementation details.



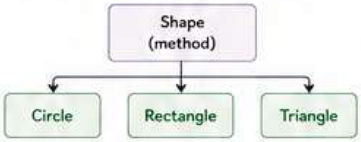
3. Inheritance

- Acquiring properties of one class into another.



4. Polymorphism

- One interface, multiple implementations.



Key Takeaways

- Class is a blueprint, object is an instance.
- OOP makes code reusable, modular and easy to maintain.
- The four pillars of OOP work together to build robust programs.

4. Constructor

- Special method used to initialize objects.
- Constructor name must be same as class name.
- No return type.

```
class Student {
  int id;
  String name;
  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }
}
```

5. this Keyword

- Used to refer current object.
- Resolves ambiguity between instance variables and parameters.

```
class Student {
  int id;
  String name;
  Student(int id, String name) {
    this.id = id;
    this.name = name;
  }
  void display() {
    System.out.println(this.id + " " + this.name);
  }
}
```

6. Access Modifiers

Modifier	Class	Package	Subclass	World
private	Yes	No	No	No
(default)	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

CHAPTER 8

Inheritance, Polymorphism & Abstraction

1. Inheritance

- Inheritance is a mechanism in which one class acquires the properties of another class.
- It promotes code reusability.

Syntax

```
class ChildClass extends ParentClass {
    // body
}
```

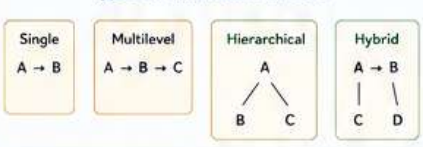
Example

```
class Animal {
   void eat() {
      System.out.println("Animal is eating");
   }
}
class Dog extends Animal {
   void bark() {
      System.out.println("Dog is barking");
   }
}

public class Main {
   public static void main(String[] args) {
     Dog d = new Dog();
     d.eat();    // inherited method
     d.bark();   // own method
   }
}
```

```
Output:
Animal is eating
Dog is barking
```

Types of Inheritance in Java



Note: Java does not support Multiple Inheritance with classes. It can be achieved using interfaces.

2. Polymorphism

- Polymorphism means "many forms".
- It allows a method to behave differently based on the object that invokes it.

Types

1. Compile-time Polymorphism (Method Overloading)

- Achieved by method overloading.
- Same method name, different parameters.

```
class MathOps {
  int add(int a, int b) { return a + b; }
  int add(int a, int b, int c) { return a + b + c; }
  double add(double a, double b) { return a + b; }
}
```

2. Run-time Polymorphism (Method Overriding)

- Achieved by method overriding.
- Occurs in inheritance.

```
class Animal {
  void sound() {
     System.out.println("Animal sound");
  }
}
class Dog extends Animal {
  @Override
  void sound() {
    System.out.println("Bark");
  }
}
public class Main {
  public static void main(String[] args) {
    Animal a = new Dog();    // upcasting
    a.sound();               // Bark
  }
}
```

Key Takeaways

- Inheritance provides reusability.
- Polymorphism allows flexibility.
- Method overloading is compile-time, method overriding is run-time.

3. Abstraction

- Abstraction hides implementation details and shows only essential features.
- Achieved using abstract classes and interfaces.

A. Abstract Class

- A class declared with 'abstract' keyword.
- Can have abstract and non-abstract methods.

```
abstract class Shape {
  abstract double area();    // abstract method
  void display() {
     System.out.println("This is a shape");
  }
}
class Circle extends Shape {
  double radius;
  Circle(double r) { radius = r; }
  double area() { return 3.14 * radius * radius; }
}
```

B. Interface

- A blueprint of a class.
- All methods are public abstract by default.
- All variables are public static final by default.

```
interface Drawable {
  void draw();
}

class Rectangle implements Drawable {
  public void draw() {
     System.out.println("Drawing Rectangle");
  }
}
```

Key Takeaways

- Abstraction reduces complexity.
- Use abstract class when you want to provide some implementation.
- Use interface to define a contract for behavior.

CHAPTER 9 — Exception Handling

1. What is Exception?

- An exception is an event that disrupts the normal flow of a program.
- In Java, exceptions are objects that represent error conditions.

Benefits

- ✓ Separates error handling code from normal code
- ✓ Improves code readability
- ✓ Helps in maintaining the program

2. Types of Exceptions

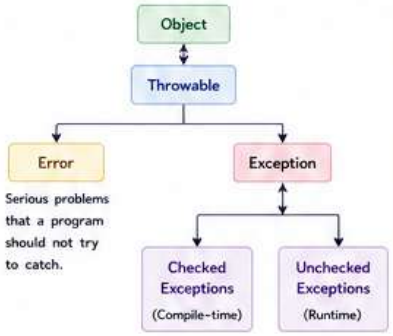
A. Checked Exceptions

- Checked at compile time.
- Must be handled or declared.
- Examples: IOException, SQLException

B. Unchecked Exceptions

- Checked at runtime.
- Arise due to programming errors.
- Examples: ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException

3. Exception Hierarchy



4. try-catch Block

```
try {
    // risky code
} catch (ExceptionType e) {
    // handling code
}
```

5. Example

```
public class Demo {
  public static void main(String[] args) {
    try {
      int a = 10, b = 0;
      int c = a / b;    // may throw exception
      System.out.println("Result: " + c);
    } catch (ArithmeticException e) {
      System.out.println("Cannot divide by zero!");
    }
    System.out.println("Program continues...");
  }
}
```

Output:

```
Cannot divide by zero!
Program continues...
```

6. try-catch-finally

```
try {
    // risky code
} catch (ExceptionType e) {
    // handling code
} finally {
    // always executed
}
```

7. throw Keyword

- Used to explicitly throw an exception.
- Useful to raise custom errors.

```
throw new ArithmeticException("Invalid input");
```

8. throws Keyword

- Used in method signature to declare exceptions.
- Transfers the responsibility of handling to the caller.

```
void method() throws IOException { ... }
```

9. Custom Exception

- We can create our own exception by extending Exception class.

```
class MyException extends Exception {
   MyException(String msg) {
      super(msg);
   }
}
```

Example:

```
if(age < 18) {
   throw new MyException("Age must be 18+");
}
```

10. Common Built-in Exceptions

Exception	Description
ArithmeticException	Arithmetic error (e.g., divide by zero)
NullPointerException	Using null reference
ArrayIndexOutOfBoundsException	Invalid array index
NumberFormatException	Invalid number format
IOException	Input/Output operation failed
ClassNotFoundException	Class not found
SQLException	Database access error

11. Best Practices

- ✓ Handle exceptions specifically.
- ✓ Use meaningful messages.
- ✓ Always release resources in finally block or use try-with-resources.
- ✓ Avoid catching generic Exception.

Key Takeaways

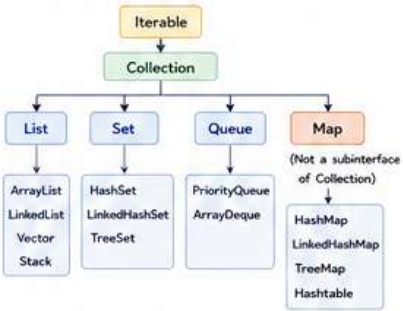
- Exception handling prevents abnormal termination.
- Use try-catch to handle exceptions gracefully.
- finally block executes always.
- Create custom exceptions for application-specific errors.
- Good exception handling improves robustness of the program.

CHAPTER 10 — Collections

1. What is Collections Framework?

- Collection in Java is a framework that provides an architecture to store and manipulate a group of objects.
- It provides interfaces and classes to perform operations like add, remove, search, sort, etc.

2. Collection Hierarchy



3. List Interface (Ordered, Duplicates Allowed)

Class	Description
ArrayList	Resizable array implementation. Fast random access. Not synchronized.
LinkedList	Doubly linked list. Good for frequent insertions and deletions.
Vector	Similar to ArrayList but synchronized (legacy).
Stack	LIFO stack. Extends Vector.

Common Methods

add(E e)	– Adds element
get(int index)	– Returns element at index
set(int index, E e)	– Updates element at index
remove(int index)	– Removes element
size()	– Returns number of elements
isEmpty()	– Checks if collection is empty
contains(Object o)	– Checks if element exists

4. Set Interface (Unique Elements)

Class	Description
HashSet	Does not maintain order. Allows null. Based on Hash table.
LinkedHashSet	Maintains insertion order. Slightly slower than HashSet.
TreeSet	Maintains sorted (natural order). Does not allow null.

Common Methods

add(E e)	– Adds element
remove(Object o)	– Removes element
contains(Object o)	– Checks if element exists
size()	– Returns number of elements

5. Queue Interface (FIFO)

Class	Description
PriorityQueue	Elements ordered by priority (natural order or comporator).
ArrayDeque	Resizable array implementation of deque. Can be used as stack or queue.

Common Methods

offer(E e)	– Inserts element
poll()	– Retrieves and removes head
peek()	– Retrieves head without removing
isEmpty()	– Checks if queue is empty

6. Map Interface (Key-Value Pairs)

Class	Description
HashMap	No order guarantee. Allows one null key and multiple null values.
LinkedHashMap	Maintains insertion order.
TreeMap	Sorted by keys (natural order). Does not allow null key.
Hashtable	Synchronized. Legacy class.

Common Methods (Map)

put(K key, V value)	– Adds key-value pair
get(Object key)	– Returns value for key
remove(Object key)	– Removes key-value pair
containsKey(Object k)	– Checks if key exists
containsValue(Object v)	– Checks if value exists
keySet()	– Returns set of keys
values()	– Returns collection of values
entrySet()	– Returns set of key-value pairs

7. Examples

ArrayList Example

```
List<String> list = new ArrayList<>();
list.add("Java");
list.add("Python");
list.add("C++");
System.out.println(list);
// [Java, Python, C++]
```

HashSet Example

```
Set<Integer> set = new HashSet<>();
set.add(10);
set.add(20);
set.add(10);    // duplicate
System.out.println(set);
// [10, 20]
```

HashMap Example

```
Map<Integer, String> map = new HashMap<>();
map.put(1, "One");
map.put(2, "Two");
map.put(3, "Three");
System.out.println(map.get(2)); // Two
```

PriorityQueue Example

```
Queue<Integer> pq = new PriorityQueue<>();
pq.offer(30);
pq.offer(10);
pq.offer(20);
System.out.println(pq.poll()); // 10
```

Key Takeaways

- Collections framework provides ready-made data structures.
- Use List when order and duplicates are important.
- Use Set when only unique elements are required.
- Use Queue for FIFO processing.
- Use Map for key-value pair storage.

CHAPTER 11

Multithreading

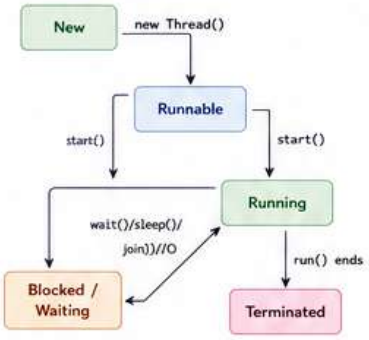
1. What is Multithreading?

- Multithreading is the ability of a program to execute multiple threads concurrently.
- It helps in maximizing CPU utilization and improving performance.

Benefits

- ✓ Better performance
- ✓ Responsiveness
- ✓ Efficient resource utilization
- ✓ Simpler code (for concurrent tasks)

2. Thread Lifecycle



3. Creating a Thread (Extending Thread class)

```
class MyThread extends Thread {
  public void run() {
    for(int i = 1; i <= 5; i++) {
      System.out.println("Thread running: " + i);
      try { Thread.sleep(500); } catch(Exception e) {}
    }
  }
}
public class Main {
  public static void main(String[] args) {
    MyThread t = new MyThread();   // creating thread
    t.start();                     // starting thread
  }
}
```

Output:
```
Thread running: 1  Thread running: 2
Thread running: 3  Thread running: 4
Thread running: 5
```

4. Creating a Thread (Implementing Runnable interface)

```
class MyRunnable implements Runnable {
  public void run() {
    for(int i = 1; i <= 5; i++) {
      System.out.println("Runnable running: " + i);
      try { Thread.sleep(500); } catch(Exception e) {}
    }
  }
}
public class Main {
  public static void main(String[] args) {
    MyRunnable r = new MyRunnable();
    Thread t = new Thread(r);   // pass runnable object
    t.start();                  // starting thread
  }
}
```

5. Common Thread Methods

Method	Description
start()	Starts the execution of the thread.
run()	Contains the task to be performed.
sleep(ms)	Pauses the thread for given milliseconds.
join()	Waits for this thread to die.
yield()	Gives a chance to other threads.
isAlive()	Checks if the thread is alive.
setPriority(p)	Sets priority (1 to 10).
getPriority()	Returns the priority.

6. Synchronization

- Used to prevent thread interference.
- Achieved using synchronized keyword.

```
class Counter {
  int count = 0;
  public synchronized void increment() {
    count++;
  }
}
```

7. Inter-Thread Communication

- Threads can communicate using wait(), notify(), notifyAll().
- Must be used inside synchronized block.

```
synchronized(lock) {
  lock.wait();      // releases lock and waits
  lock.notify();    // wakes up one waiting thread
  lock.notifyAll(); // wakes up all waiting threads
}
```

8. Daemon Thread

- A background thread that supports user threads.
- JVM terminates daemon threads when all user threads finish.

```
Thread t = new Thread(() -> {
  while(true) {
    System.out.println("Daemon thread running...");
  }
});
t.setDaemon(true);
t.start();
```

9. Thread Priorities

- Priority range: 1 (MIN_PRIORITY) to 10 (MAX_PRIORITY)
- Default priority is 5 (NORM_PRIORITY)

Constant	Value
Thread.MIN_PRIORITY	1
Thread.NORM_PRIORITY	5
Thread.MAX_PRIORITY	10

Key Takeaways

- Multithreading improves performance and responsiveness.
- Threads can be created by extending Thread class or implementing Runnable interface.
- Use synchronization to avoid data inconsistency.
- Use wait(), notify(), notifyAll() for communication.
- Choose correct priority and make threads daemon when needed.

CHAPTER 12

Mini Projects + Interview Questions

A. Mini Projects

1. Student Management System

Console application to add, view, update and delete student records.

Concepts Used: OOP, ArrayList, File I/O, Exception Handling

2. Bank Account System

Create a bank system with Account creation, deposit, withdraw and balance check.

Concepts Used: OOP, Encapsulation, Exception Handling

3. Library Management System

Add books, issue books, return books and search books.

Concepts Used: Collections, File Handling, OOP

4. To-Do List Application

Console based To-Do List to add, remove, update and view tasks.

Concepts Used: ArrayList, Date & Time, OOP

5. Password Generator

Generate strong random passwords based on user preferences.

Concepts Used: Strings, Random class, StringBuilder

Project Tips

- ✓ Break the problem into small modules.
- ✓ Use proper classes and methods.
- ✓ Handle exceptions and invalid inputs.
- ✓ Follow good coding practices and commenting.
- ✓ Try to add file persistence for data.

1 Core Java – Basics

Q1. What is JVM?
Ans. JVM (Java Virtual Machine) is an abstract machine that runs Java bytecode.

Q2. What are the components of JDK?
Ans. JDK = JRE + Development tools (javac, java, javadoc, etc.)

Q3. What is the difference between JRE and JVM?
Ans. JRE is an implementation that provides running environment. JVM is a part of JRE that executes bytecode.

2 OOP Concepts

Q1. What is Encapsulation?
Ans. Binding data and methods together and restricting direct access to some components.

Q2. What is Inheritance?
Ans. Acquiring properties and behavior of one class into another class.

Q3. What is Polymorphism?
Ans. Ability to take many forms. It is achieved by method overloading and overriding.

3 Exception Handling

Q1. What is an exception?
Ans. An event that disrupts the normal flow of program.

Q2. What is the difference between throw and throws?
Ans. throw is used to throw an exception explicitly. throws is used in method signature to declare exception.

Q3. What is finally block?
Ans. It always executes whether exception occurs or not.

4 Collections Framework

Q1. What is Collection in Java?
Ans. Collection is a framework that provides architecture to store and manipulate group of objects.

Q2. What is the difference between List and Set?
Ans. List maintains insertion order and allows duplicates. Set does not allow duplicates.

Q3. What is HashMap?
Ans. HashMap stores key-value pairs and allows one null key and multiple null values.

5 Multithreading

Q1. What is a thread?
Ans. A thread is a lightweight sub-process within a process.

Q2. What is the difference between sleep() and yield()?
Ans. sleep() pauses the thread for given time. yield() gives chance to other threads.

Q3. How do you achieve synchronization?
Ans. By using synchronized keyword or methods.

6 Miscellaneous

Q1. What is abstraction?
Ans. Hiding implementation details and showing only necessary features.

Q2. What is the difference between == and equals()?
Ans. == compares references, equals() compares content.

Q3. What is garbage collection?
Ans. Automatic process of reclaiming memory by JVM.

General Tips for Interviews

- ✓ Understand concepts clearly.
- ✓ Write clean and readable code.
- ✓ Think before you code.
- ✓ Handle edge cases.
- ✓ Practice coding problems regularly.